

გაკვეთილი 10 mod 3 (3.2)

FOR ციკლი JavaScript-ში

რა არის `for`

`for` ციკლი გამოიყენება მაშინ, როცა გვინდა ერთი და იგივე მოქმედება რამდენჯერმე შესრულდეს.

მაგალითად:

- 1-დან 10-მდე რიცხვების დაბეჭდვა
- ტექსტის 5-ჯერ გამეორება
- რიცხვების ჯამის დათვლა

`for` განსაკუთრებით კარგია მაშინ, როცა ვიცით რამდენჯერ უნდა შესრულდეს კოდი.

`for` ციკლის სინტაქსი

```
for (initialization; condition; update) {  
  // კოდი  
}
```

ეს სამი ნაწილი ასეთია:

1. `initialization`

აქ ვქმნით საწყის ცვლადს.

მაგალითად:

```
let i = 0
```

ანუ ციკლი იწყება `i = 0`-დან.

2. `condition`

ეს არის პირობა — სანამ ეს პირობა მართალია, ციკლი იმუშავებს.

მაგალითად:

`i < 5`

ანუ სანამ `i` არის 5-ზე ნაკლები, ციკლი გაგრძელდება.

3. update

ყოველი გამეორების შემდეგ ცვლადი იცვლება.

მაგალითად:

`i++`

ანუ `i` ყოველ ჯერზე 1-ით გაიზრდება.

პირველი მაგალითი

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

როგორ მუშაობს

- თავიდან `i = 0`
- მოწმდება: `i < 5` → `true`
- იბეჭდება `0`
- მერე `i++` → `i = 1`

შემდეგ ისევ:

- `1 < 5` → `true`
- იბეჭდება `1`

და ასე გრძელდება.

Output

0
1
2
3
4

რატომ არ დაიბეჭდა 5?

იმიტომ რომ პირობა იყო:

`i < 5`

ანუ `i` უნდა იყოს **5-ზე** ნაკლები.

როცა `i` გახდა 5, პირობა უკვე `false` გახდა და ციკლი გაჩერდა.

მაგალითი: 1-დან 5-მდე

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

ახსნა

აქ დავიწყეთ 1-დან და ვაგრძელებთ სანამ `i <= 5`.

Output

1
2
3
4
5

მაგალითი: 10-დან 1-მდე

```
for (let i = 10; i >= 1; i--) {  
  console.log(i);  
}
```

ახსნა

აქ:

- იწყება 10-დან
- პირობაა `i >= 1`

- ყოველ ჯერზე `i--` ანუ 1-ით მცირდება

Output

10
9
8
7
6
5
4
3
2
1

მაგალითი: ლუწი რიცხვები

```
for (let i = 0; i <= 10; i += 2) {  
  console.log(i);  
}
```

ახსნა

აქ `i += 2` ნიშნავს, რომ ცვლადი ყოველ ჯერზე 2-ით იზრდება.

მნიშვნელობები იქნება:

- 0
- 2
- 4
- 6
- 8
- 10

Output

0
2
4
6

8
10

მაგალითი: ტექსტის გამეორება

```
for (let i = 0; i < 3; i++) {  
  console.log("Hello");  
}
```

ახსნა

ციკლი შესრულდება 3-ჯერ.

Output

Hello
Hello
Hello

მაგალითი: ჯამის გამოთვლა

```
let sum = 0;  
  
for (let i = 1; i <= 5; i++) {  
  sum += i;  
}
```

```
console.log(sum);
```

ახსნა

თავიდან:

sum = 0

შემდეგ:

- `sum = 0 + 1 → 1`
- `sum = 1 + 2 → 3`
- `sum = 3 + 3 → 6`
- `sum = 6 + 4 → 10`

- `sum = 10 + 5` → 15

Output

15

მაგალითი: კვადრატები

```
for (let i = 1; i <= 5; i++) {  
  console.log(i * i);  
}
```

Output

1
4
9
16
25

მაგალითი: ტექსტთან ერთად

```
for (let i = 1; i <= 5; i++) {  
  console.log("Number: " + i);  
}
```

Output

Number: 1
Number: 2
Number: 3
Number: 4
Number: 5

FOR-EACH JavaScript-ში

JavaScript-ში ზუსტად `for-each` კლასიკური სიტყვა არ იწერება ისე, როგორც ზოგ სხვა ენაში, მაგრამ იგივე იდეა გვაქვს ორი გზით:

- `for...of`
- `forEach()`

ორივე გამოიყენება, როცა გვინდა მასივის ელემენტებზე გადავიაროთ.

1. `for...of`

სინტაქსი

```
for (let item of array) {  
  // კოდი  
}
```

აქ `item` არის მასივის თითოეული ელემენტი.

მაგალითი

```
let numbers = [5, 7, 10];  
  
for (let n of numbers) {  
  console.log(n);  
}
```

ახსნა

მასივია:

`[5, 7, 10]`

ციკლი სათითაოდ აიღებს:

- ჯერ 5-ს
- მერე 7-ს
- მერე 10-ს

Output

5
7
10

მაგალითი: სახელები

```
let names = ["Giorgi", "Ana", "Nika"];
```

```
for (let name of names) {  
  console.log(name);  
}
```

Output

Giorgi

Ana

Nika

მაგალითი: ჯამის პოვნა მასივში

```
let numbers = [1, 2, 3, 4, 5];
```

```
let sum = 0;
```

```
for (let n of numbers) {  
  sum += n;  
}
```

```
console.log(sum);
```

ახსნა

- `sum = 0`
- მერე ემატება 1
- მერე 2
- მერე 3
- მერე 4
- მერე 5

ანუ:

$1 + 2 + 3 + 4 + 5 = 15$

Output

15

მაგალითი: მაქსიმუმის პოვნა

```
let numbers = [3, 10, 5];  
let max = numbers[0];
```

```
for (let n of numbers) {  
  if (n > max) {  
    max = n;  
  }  
}
```

```
console.log(max);
```

ახსნა

თავიდან:

`max = 3`

შემდეგ:

- `n = 3` → `3 > 3?` არა
- `n = 10` → `10 > 3?` კი → `max = 10`
- `n = 5` → `5 > 10?` არა

Output

10

მაგალითი: ყველა ელემენტის გაზრდა

```
let numbers = [1, 2, 3];
```

```
for (let n of numbers) {  
  console.log(n + 1);  
}
```

Output

2

3

2. `forEach()`

ეს არის მასივის მეთოდი.

სინტაქსი

```
array.forEach(function(item) {  
  // კოდი  
});
```

ან მოკლედ:

```
array.forEach(item => {  
  // კოდი  
});
```

მაგალითი

```
let numbers = [5, 7, 10];  
  
numbers.forEach(function(n) {  
  console.log(n);  
});
```

Output

```
5  
7  
10
```

იგივე `arrow function`-ით

```
let numbers = [5, 7, 10];  
  
numbers.forEach(n => {  
  console.log(n);  
});
```

Output

5
7
10

მაგალითი: ტექსტის ელემენტები

```
let fruits = ["apple", "banana", "orange"];
```

```
fruits.forEach(fruit => {  
  console.log(fruit);  
});
```

Output

apple
banana
orange

მაგალითი: ჯამის პოვნა

```
let numbers = [1, 2, 3, 4];  
let sum = 0;
```

```
numbers.forEach(n => {  
  sum += n;  
});
```

```
console.log(sum);
```

Output

10

for და **for...of** განსხვავება

for

გამოგადგება როცა:

- გვინდა ზუსტად ვიცოდეთ რამდენჯერ იმუშავოს
- გვინდა ინდექსით მუშაობა
- გვინდა 1-დან 10-მდე, 10-დან 1-მდე, ნაბიჯით, უკუღმა

მაგალითი:

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

for...of

გამოგადგება როცა:

- უკვე გვაქვს მასივი
- გვინდა მასივის ელემენტებზე პირდაპირ გავლა

მაგალითი:

```
let names = ["Ana", "Nika", "Gio"];  
  
for (let name of names) {  
  console.log(name);  
}
```

როდის რომელი ჯობს?

- თუ მუშაობ რიცხვების დიაპაზონზე → `for`
- თუ მუშაობ მასივზე → `for...of` ან `forEach()`

სავარჯიშოები — FOR

მარტივი

1. დაბეჭდე 1-დან 10-მდე რიცხვები.
2. დაბეჭდე 1-დან 20-მდე რიცხვები.

3. დაბეჭდე 10-დან 1-მდე რიცხვები.
 4. დაბეჭდე ყველა ლუწი რიცხვი 0-დან 20-მდე.
 5. დაბეჭდე "Hello" 5-ჯერ.
 6. დაბეჭდე "JavaScript" 3-ჯერ.
 7. დაბეჭდე 1-დან 10-მდე რიცხვების კვადრატები.
 8. დაბეჭდე 1-დან 10-მდე რიცხვების კუბები.
 9. იპოვე 1-დან 100-მდე რიცხვების ჯამი.
 10. იპოვე 1-დან 50-მდე ყველა ლუწი რიცხვის ჯამი.
-

სავარჯიშოები — FOR...OF / FOREACH

11. დაბეჭდე მასივის ყველა ელემენტი:

[2, 4, 6, 8]

12. იპოვე ჯამი:

[1, 2, 3, 4, 5]

13. იპოვე მაქსიმალური ელემენტი:

[10, 5, 20, 3]

14. იპოვე მინიმალური ელემენტი:

[10, 5, 20, 3]

15. დაბეჭდე ყველა სახელი:

["Giorgi", "Ana", "Nika"]

16. დაბეჭდე ყველა სახელი დიდი ასოებით.

17. დაითვალე რამდენი ლუწი რიცხვია მასივში.

18. დაითვალე რამდენი კენტი რიცხვია მასივში.

19. დაბეჭდე მხოლოდ 10-ზე მეტი რიცხვები მასივიდან.

20. იპოვე მასივის საშუალო მნიშვნელობა.

1. რიცხვების ჯამი (for ციკლი + if)

მომხმარებელმა შეიყვანოს რიცხვი `n`.

დაბეჭდე 1-დან `n`-მდე ყველა რიცხვის ჯამი.

👉 დამატებით:

დაბეჭდე ცალკე მხოლოდ ლუწი რიცხვების ჯამი.

2. ლუწი და კენტი რიცხვების დათვლა (**while + if**)

მომხმარებელი შეიყვანს რიცხვებს (მაგალითად, 5-ჯერ).

დათვალე რამდენი არის:

- ლუწი
- კენტი

👉 ბოლოს დაბეჭდე ორივე რაოდენობა.

3. ფაქტორიალი (**for ან while**)

მომხმარებელმა შეიყვანოს რიცხვი n .

დაითვალე მისი ფაქტორიალი:

$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$

👉 შედეგი დაბეჭდე.

4. ყველაზე დიდი რიცხვის პოვნა (**if + ციკლი**)

მომხმარებელმა შეიყვანოს 5 რიცხვი.

იპოვე მათ შორის უდიდესი რიცხვი.

👉 მინიშნება:

შეინახე ერთი ცვლადი და შეადარე ყოველ ახალ რიცხვს.

5. გამრავლების ცხრილი (**for** ციკლი)

მომხმარებელმა შეიყვანოს რიცხვი n .

დაბეჭდე მისი გამრავლების ცხრილი:

მაგალითად $n = 3$:

$$3 \times 1 = 3$$

$$3 \times 2 = 6$$

...

$$3 \times 10 = 30$$

6. გამოფების დათვლა (for + if)

მომხმარებელმა შეიყვანოს რიცხვი `n`.

იპოვე:

- რამდენი გამოფი აქვს ამ რიცხვს
- და დაბეჭდე ყველა გამოფი

მაგალითად `n = 6`:

1, 2, 3, 6

👉 დამატებით:

თუ გამოფების რაოდენობა არის 2 → დაბეჭდე "Prime" (მარტივი რიცხვია)

პატარა შემაჯამებელი

for

გამოიყენება, როცა წინასწარ ვიცით გამოფრებების რაოდენობა.

for...of

გამოიყენება, როცა გვინდა მასივის ელემენტებზე პირდაპირ გადასვლა.

forEach()

იგივე იდეაა, უბრალოდ მასივის მეთოდით იწერება.